

# Lec3

## 界的继续放缩

回顾上次lecture的定理：对于任意 $n, m > 0$ 和任意有限函数 $f : \{0, 1\}^n \rightarrow \{0, 1\}^m$ ，存在一个boolean circuit  $C$ 可以计算 $f$ ，最多需要 $O(m \cdot n \cdot 2^n)$ 个门。

那么，这个界能否进一步放缩？

## 放缩到 $O(m \cdot 2^n)$

我们可以利用之前的LOOKUP的思想。

| $X[0]$ | $X[1]$ | ... | $X[n - 2]$ | $X[n - 1]$ | $Y[j]$ |
|--------|--------|-----|------------|------------|--------|
| 0      | 0      | ... | 0          | 0          | 0      |
| 0      | 0      | ... | 0          | 1          | 0      |
| 0      | 0      | ... | 1          | 0          | 1      |
| ...    | ...    | ... | ...        | ...        | ...    |

对于输出的某一位 $Y[j]$ ，其有 $2^n$ 种可能对应的输入组合，每一种输入组合的结果拼在一起就像LOOKUP函数中那一个等待查询的数组，而查询的索引就是输入变量。

例如：

| $X[0]$ | $X[1]$ | $Y[0]$ | 对应查询   |
|--------|--------|--------|--------|
| 0      | 0      | 1      | $G[0]$ |
| 0      | 1      | 0      | $G[1]$ |
| 1      | 0      | 1      | $G[2]$ |
| 1      | 1      | 1      | $G[3]$ |

实际上在做的是：

$$\text{LOOKUP}_2(G[0], G[1], G[2], G[3]; X[0], X[1])$$

由LOOKUP的实现可知，计算 $Y[j]$ 需要 $O(2^n)$ 个门，因此计算所有 $m$ 位输出需要 $O(m \cdot 2^n)$ 个门。

## 放缩到 $O(m \cdot \frac{2^n}{n})$

同样，我们先只关心输出的某一位 $Y[j]$ 。

将有 $2^n$ 行的真值表分成 $2^k$ 个子表，每个子表的行数为 $2^{n-k}$ ，记作 $T_0, T_1, \dots, T_{2^k-1}$ 。

于是，我们的查询分成了两步：第一步是确定子表，第二步是在子表中查询。

我们先来看第二步，要找到在子表的哪一行，这时我们只需要看输入变量的后 $n - k$ 位。

我们可以直接用LOOKUP实现，与之前的放缩类似，需要 $O(2^{n-k})$ 个门。

我们再来看第一步。第一步的输出实际上是长度为 $2^{n-k}$ 的字符串，即我们要找的子表的最后一列 $Y[j]$ 真值。只有获得这些真值，我们才能进行第二步的查询。

因此，第一步的输入是前 $k$ 位输入变量，输出是长度为 $2^{n-k}$ 的字符串：

$$g : \{0, 1\}^k \rightarrow \{0, 1\}^{2^{n-k}}$$

问题变成了 $g$ 需要多少个门。

如果输入的长度远远小于输出的长度，例如 $\{0, 1\}^2 \rightarrow \{0, 1\}^{100}$ ，输出的每一位和输入的对应关系是 $\{0, 1\}^2 \rightarrow \{0, 1\}$ ，有 $2^{2^2} = 16$ 种可以刻画的函数。

实际上，100位输出中，有很多位的对应关系是一样的。

那么，对于这个例子，我把16种函数都实现，这100位的输出，每一位肯定是这16种函数中挑一个运行出来。

一共16种函数，各自需要4个门（通过真值表理解），连接到输出需要100个门，因此总共需要 $4 \cdot 16 + 100 = 164$ 个门。

回到 $g$ 进行泛化：一共 $2^{2^k}$ 种函数，每个函数需要 $O(2^k)$ 个门，连接到输出需要 $2^{n-k}$ 个门，因此总共需要 $O(2^{2^k} \cdot 2^k + 2^{n-k})$ 个门。

令 $k = \log_2(n - 2 \log_2 n)$ ，则上式变为 $O(\frac{2^n}{n})$ ，且第二步也需要 $O(\frac{2^n}{n})$ 个门，因此对于单位的输出需要 $O(\frac{2^n}{n})$ 个门，所有 $m$ 位输出需要 $O(m \cdot \frac{2^n}{n})$ 个门。

## 是否还能继续放缩？

否！

我们可以证明：存在一个有限函数，至少需要 $O(m \cdot \frac{2^n}{n})$ 个门。

先只考虑 $m = 1$ 的情况： $\{0, 1\}^n \rightarrow \{0, 1\}$ ，刚刚我们已经算出来，可能的函数有 $2^{2^n}$ 种。

此外，对于一个NAND-CIRC program  $P$ ，如果它的行数不超过 $s$ ，则可以将其编码成长度为 $O(s \log s)$ 的01串。（证明在下一阶段给出）

如果上面的成立，则有行数不超过 $s$ 的NAND-CIRC program的数量不超过长度为 $Cs \log s$ 的01串的数量，其中 $C$ 为某个常数。因此，行数不超过 $s$ 的NAND-CIRC program的数量不超过 $2^{Cs \log s}$ 个。

令 $s = \frac{2^n}{Cn}$ ，则行数不超过 $s$ 的NAND-CIRC program的数量不超过 $2^{2^n}$ 个。

我们看到， $m = 1$ 的有限函数有 $2^{2^n}$ 种，而行数不超过 $s = \frac{2^n}{Cn}$ 的NAND-CIRC program的数量不超过 $2^{2^n}$ 个，因此至少存在一个函数，其对应的NAND-CIRC program的行数大于 $s = \frac{2^n}{Cn}$ ，也就是至少需要 $O(\frac{2^n}{n})$ 个门。再考虑 $m$ 位，则至少需要 $O(m \cdot \frac{2^n}{n})$ 个门。

## program编码

如何对一个program进行编码？

我们假设NAND-CIRC program，有 $n$ 个输入变量 $X[0], X[1], \dots, X[n-1]$ ， $m$ 个输出变量 $Y[0], Y[1], \dots, Y[m-1]$ ，还有若干个中间变量 $temp[0], temp[1], \dots$ 。

由于NAND-CIRC program的每一行的形式都是 $A = \text{NAND}(B, C)$ ，因此如果program有 $s$ 行，则至多有 $3s$ 个变量。

因此，我们对这 $3s$ 个变量重命名为 $var[0], var[1], var[2], \dots, var[3s-1]$ ，规定前 $n$ 个变量对应输入变量，紧接着的 $m$ 个变量对应输出变量，剩下的变量对应中间变量。

如上重命名后，program的每一行就可以简写为长度为3的三元组。例如 $(7, 2, 1)$ 表示的就是 $var[7] = \text{NAND}(var[2], var[1])$ 。

我们只需要对这些三元组进行编码。变量数量不超过 $3s$ ，因此每个变量的编码长度为 $\lceil \log(3s) \rceil$ ，一个三元组的编码长度为 $3 \lceil \log(3s) \rceil$ 。由于program有 $s$ 行，因此整个program的编码长度为 $3s \lceil \log(3s) \rceil$ 。

这个program本身可以作为输入给到其他程序。

## EVAL函数

考虑函数

$$\text{EVAL}_{s,n,m} : \{0, 1\}^{3s \lceil \log 3s \rceil + n} \rightarrow \{0, 1\}^m$$

这个函数的输入分为两部分：

- 第一部分长度为 $3s \lceil \log 3s \rceil$ ，记作 $p$ ，表示某个NAND-CIRC program的编码
- 第二部分长度为 $n$ ，记作 $x$ ，表示 $p$ 的输入

这个函数的输出就是运行 $p$ 时，输入 $x$ 所得到的输出。

$$\text{EVAL}_{s,n,m}(px) = \begin{cases} p(x) & \text{if } p \text{ is a valid encoding} \\ 0^m & \text{otherwise} \end{cases}$$

如何使用一个circuit（program）来模拟 $\text{EVAL}_{s,n,m}$ ？

对于任意固定的 $s, n, m$ ，我们都可以实现，因为这是一个有限函数，但实现它需要多少个门？

## Theorem

实现 $\text{EVAL}_{s,n,m}$ 需要 $O(s^2 \log s)$ 个门。

### "Proof"

第一步：写一个python program

第二步：将python program转换为NAND-CIRC program

```
# 初始化，V为数组
for i in range(3s):
    update(V,i,0)

# 把x赋给前n位变量
for i in range(n):
    update(V,i,xi)

# 运行p
for (i,j,k) in p:
    a=get(V,j)
    b=get(V,k)
    c=NAND(a,b)
    update(V,i,c)

# 返回输出
for j in range(m):
    yj=get(V,n+j)
return y0,...ym-1
```

get 可以用之前的LOOKUP实现，每个 get 需要 $O(s)$ 个门。

对于 update，其本质也是一个有限函数：

$$\{0,1\}^{3s+\lceil \log 3s \rceil +1} \rightarrow \{0,1\}^{3s}$$

$3s$ 是数组的长度， $\lceil \log 3s \rceil$ 是索引的长度，1是要更新的值。

若我们只看某一位：

$$\{0,1\}^{3s+\lceil \log 3s \rceil +1} \rightarrow \{0,1\}$$

$$g_j(V,i,a) = \begin{cases} a & \text{if } j = i \\ V[j] & \text{otherwise} \end{cases}$$

实现一个单一的 $g_j$ 需要 $O(\log s)$ 个门。（判断两个长度为 $n$ 的串是否相等，需要 $O(n)$ 个门，待证明）

然而这只是考虑了一位的情况。若 $3s$ 位都考虑，则需要 $O(s \log s)$ 个门。

一共进行了 $O(s)$ 次 get 和 update，因此总的门数为 $O(s^2 \log s)$ 。